

Medical Image Processing and Machine Learning

VU 193.215

Assignment 2: Bone Segmentation

Computer Vision Lab
Institute of Visual Computing and Human-Centered Technology

georg.langs@tuwien.ac.at

roxane.licandro@tuwien.ac.at

philipp.seeboeck@meduniwien.ac.at

Tutors: Emilia Mc Greal, Benno Steinegger

emilia.mc.greal@tuwien.ac.at

benno.steinegger@tuwien.ac.at

SS 2026

General

This document summarizes the tasks and technical requirements for assignment 2 of the exercise (UE) *Medical Image Processing* at TU Wien in the summer term 2026.

Due: 20.06.2026, 23:59 (CET) - one submission per team via TUWEL

Mandatory group interviews - IN PERSON: 25.06.2026 - 09:00 - 13:25 (CET) (20 min slots)

Total achievable points: 55

In the summer term 2026 the tasks of assignment 2 have to be implemented using Python3.9 and Jupyter Notebook as well. Create a new virtual Python environment using the yml file with the new dependencies provided (see description assignment 1 for installation instructions of virtual environments).

Usage of LLMs

The use of any Large Language Models (LLMs), including but not limited to ChatGPT, is **strictly prohibited**.

Detected plagiarism or any evidence of LLM use in completing this assignment will be considered a serious violation of academic integrity and result in extensive point reduction and an invitation to a formal interview to check the submission and knowledge of the group and students.

Deliverables

- Executable code as `.zip`, filename: `Submission-BoneSeg-XX.zip`, where `XX` holds your group number. Also include the necessary files to run the code in the notebook. Also include the necessary files to run the code in the notebook.
- The python notebook `notebook.ipynb` with the requested changes, which computes all results and plots without user interaction. The notebook should also contain the group number, the name, and the student registration number (Matrikelnummer) of all team members. Please also rename the `notebook.ipynb` to `notebook_XX.ipynb`, where `XX` holds your group number.
- The code has to be documented and commented, explaining the reasoning behind your implementation.
- **A PDF report** that contains all theoretical answers and interpretations of all questions (see sections 2.3 and 3.5; max 8 pages, including figures and references).
 - The report's front page contains group number and team member names and student registration number (Matrikelnummer).
 - The report has to be short and precise (do not include code). Please answer only the theoretical questions and do not describe your implementation, unless otherwise stated.
 - The report can be written in German or English.

NOTE: We will provide support for issues that might arise with your Miniconda setup, but if you have a valid reason not to use it or if you have problems setting up your environment in general, use the TUWEL exercise forums or contact one of the tutors.

Assignment 2

Aim of the second assignment is the implementation of a segmentation algorithm for bone contours. We will investigate image features extraction and selection methods as bases for image segmentation. Three different approaches are investigated, (1) a classifier based approach using Random Forests, (2) a Deep Learning approach using a UNet and (3) a bone contour Shape Model based approach that uses PCA resulting in a simplified version of Particle Filters¹.

Relevant topics within this exercise:

- Feature extraction
- Classification and feature selection using Random Forests
- Implementation and optimization of a cost function for segmentation

Data and helper functions are provided within the provided source code and documentation:

Helper functions (*helper_functions.py*)

- `get_data` loads all the necessary variables that you will be working on
- `plot_shape` can be used to plot generated shapes and the given mean shape
- `plot_convolutions` plots images along with their convolutions
- `plot_prediction_triplets` plots triplets of images, segmentation predictions and segmentation ground truths
- `show_feature_importance` visualizes the importance of a Random Forest's features
- `evaluate_binary_segmentation` computes quantitative measures between a predicted and target segmentation for evaluating binary segmentation performance using a Confusion Matrix, Dice Score, Precision and Recall.
- `optimize` is used to optimize parameters with a given cost function f
- `plot_fitted_shapes` plots fitted shapes together with a segmentation and optionally with the ground truth landmarks

Additional provided functions - you do not need to change these

- ***training.py*** containing `train_unet_model` trains a U-Net and returns the trained model
- ***model.py*** containing the model to be trained
- ***optimize_demo.py*** containing a small demonstration of how the `optimize` function can be used

Dataset (*handdata.mat*)

The dataset contains the following sets:

- `images` - contains the images visualising bones
- `masks` - contains corresponding image contour masks of the bone
- `landmarks` - contains sets of landmark coordinates

¹ see PDF included in the files "Particle Filters MICCAI 2004.pdf"

- `aligned` - contains already aligned landmarks of each bone of the PCA

Useful Python Packages for this assignment

matplotlib.pyplot – imported as `plt` in notebook.ipynb

numpy – imported as `np` in notebook.ipynb

scikit-learn and scikit-image – imported as `sklearn` and `skimage` in notebook.ipynb

Tasks

Maximum number of points of subtasks are denoted in brackets (55 in total). Images 1-30 are part of the training data (PCA model and classification training), images 31-50 are part of the test data to evaluate your implementation.

1. Data Exploration (10 points)

Coding Tasks (10 points):

1.1 Plot the first 5 bone images stored in `images` and the respective landmarks stored in `landmarks` on top. **(2 points)**

1.2 Plot one set of landmarks together with the corresponding set of aligned landmarks. **(1 point)**

1.3 Extend your function `generate_shape` from Assignment 1 so that it allows scaling, rotation and translation of shapes according to the parameters `scaling`, `rotation`, `x_t`, `y_t` (hint: **rotation matrix**). **(4 points)**

Next to the parameters needed for PCA, the extended function should have 4 additional parameters:

```
def generate_shape(b, eigen_vectors, mean_shape, scaling,
rotation, x_t, y_t):
```

1.4 Perform PCA on the first 30 landmark sets in `aligned`. Use your *group number* assigned in TUWEL for parameterizing the following transformations, apply these to the bone shapes and plot them with the mean shape using `plot_shape`. **(3 points)**

- rotate by $(90 + 2 * [\textit{group number}])$ degrees
- move + $(90 + [\textit{group number}])$ pixels in x direction and -50 in y direction
- scale the bone by a factor 1.5
- use a PCA weight vector of your choice

Apply the transformation and plot the shape and the `mean_shape` using `plot_shape`

No interpretation questions for this task.

2. Edge Detection via Convolutions and Feature Extraction (20 Points)

The aim of this task is to discover filter based and non-filter based image feature extraction methods.

Coding Tasks (18 points):

2.1 Edge Detection via Convolutions (8 points)

- a) Create 2 edge detection kernels (image filters) represented as matrices (F_x , F_y) of dimension $[i, j]$ where i and j are 3. We will focus on the filters called Prewitt Operator. These have the aim of identifying edges in images by focusing on detecting changes in intensity levels. You should implement a horizontal and vertical edge detection filter **(2 points)**:

Horizontal Prewitt Operator F_x

1	0	-1
1	0	-1
1	0	-1

Vertical Prewitt Operator F_y

-1	-1	-1
0	0	0
1	1	1

You may also create additional kernels for edge detection, if you want to experiment with these.

- b) Implement the convolution operation on your own as a function `conv2d(image, kernel)` with the two input parameters `image`, representing a 2D image of size $[m, n]$ and `kernel`, representing a 3×3 filter kernel. The output should be the convolved image of size $[m, n]$. Use zero-padding for convolving the kernel at the image boundaries. **(5 points)**

The function should perform the following equation on the inputs to perform the convolution:

$$O(m,n) = (I \oplus F)(m,n) = \sum_i \sum_j I(m-i, n-j) F(i,j)$$

where $\oplus$ denotes the convolution operator, I the input image of size $[m,n]$, F the filter kernel of size $[i,j]$ and O the output image of size $[m,n]$. A schematic illustration of the convolution process is also illustrated in Figure 1.

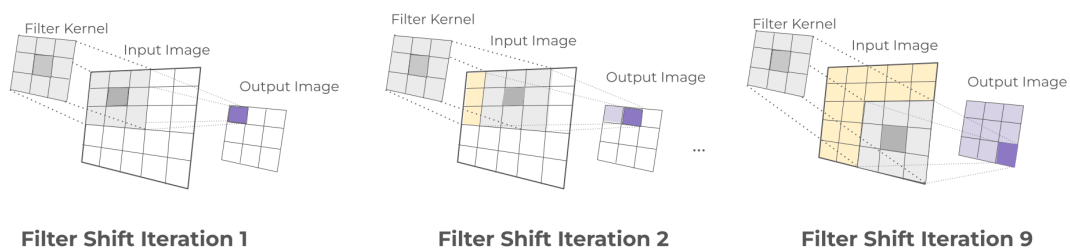


Figure 1 Schematic illustration of image convolution

- c) Select 3 random bone images of the dataset `images` loaded and convolve each of them with the filters you created. Visualise the resulting images using the function `plot_convolutions`. **(1 point)**

2.2 Image Feature Computation (10 points)

Implement a function `computeFeatures(image)` that returns a feature matrix of size `[nfeatures × npixels]` of an image, including the following `n` features:

- Grey value of an image
- Gradient in x- and y- direction
- Magnitude of the gradient
- x- and y- coordinates of a pixel

Illustrate the features of the first image in `images`. Feel free to implement and evaluate other additional features!

Visualize the features for the first image in `images`.

2.3 Interpretation Questions of Edge Detections (2 points):

1. Discuss what you observe in the output images of `plot_convolutions` from coding task 2.1 c), and explain the impact of the different filter operators in 2-3 sentences. **(2 points)**

3. Classification and Feature Selection (25 points)

In this task we investigate how image features are selected to train a classifier to solve a binary classification task - the detection of edges (bone contours) in an image.

The classification result provides per image pixel a label, where 1 denotes an edge and 0 background/no edge. In total in this tasks we will compare and explore three bone contour extraction methods: (1) Random Forest², (2) a U-Net³ based Deep Learning framework (with residual connections instead of concatenations in the decoder stream) and (3) Particle Filters using a defined training and testing set.

Coding Tasks (15.5 points):

3.1 Data Preparation

Split the image set of 50 images and corresponding masks into a training set of 30 images (`I_train`) and masks (`y_train`) and a test set of 20 images (`I_test`) and corresponding masks (`y_test`). Compute the features of all images and store them in `X_train` and `X_test`, respectively.

3.2 Random Forest (RF) (7 points)

Train and evaluate a Random Forest Classifier.

² https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm [last accessed 2026-05-27]

³ https://doi.org/10.1007/978-3-319-24574-4_28 [last accessed 2026-05-23]

- a) Implement the function `train_rf(I_train, y_train)`, taking as input a training image dataset `I_train` and corresponding set of masks `y_train` encoding the location of bone contours (target labels).

The function returns the trained Random Forest classifier. Use the Random Forest classifier (`sklearn.ensemble.RandomForestClassifier`) for this purpose which is implemented in the Python package `scikit-learn`⁴. Get familiar with the parameters and select suitable ones for the training.

The RF classifier should be trained on **extracted image features** and not the images directly. Thus, compute image features for all input images in `I_train` and `I_test` and store the resulting features in `X_train` and `X_test`. **(3 points)**

Hint: To speed up the training process you should use all pixels of the bone contours but only a randomly sampled subset of the background pixels (equal amount of foreground (bone contours) and background (non-bone contours)).

- b) Use the image features `X_train` as input to train your Random Forest classifier and the corresponding image masks `y_train` as class labels. **(1 point)**
- c) Implement the function `predict_segmentation_rf()` which takes the Random Forest classifier as well as an image and returns the prediction from the classifier. Visualize the prediction of the first test image (from `I_test`) and compare it to the corresponding target label from `y_test`. **(1 point)**
- d) Use the trained classifier and predict bone contours for the feature test set `X_test` and store the predictions in `y_pred_RF` using `plot_prediction_triplets`. Compare the bone contour predictions `y_pred_RF` to the target labels `y_test` using the helper function `evaluate_binary_segmentation` to get Dice Score, Precision and Recall as similarity metric. **(2 points)**

3.3 U-net (2.5 points)

In this task we will train two U-Nets – one with data augmentation and one without to evaluate the results and compare them to the Random Forest output.

- a) Train two U-Net bone contour classifiers with the helper function `train_model` – one with the flag `augment=False` and the other with `augment=True`. **(0.5 points)**
Note: The U-Net classifier trains on the **training images** directly with the corresponding image masks as labels. The U-Net approach learns the relevant image features for classification in the training process directly, thus no precomputation of image features is required here.
The training function requires as input all images but splits them into training, validation and testing (30-10-10). After training, the function will plot training and validation loss.
- b) Instead of using a predefined set of filters (as we used in Task 2.1 for edge detection), a convolutional neural network learns the filter weights during training to

⁴ <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
[last accessed 2026-05-27]

extract image features in combination with convolution. In this task we want to analyse and visualize which image features are extracted by the first layer of one of the trained U-Net classifiers. Therefore, use the trained U-Net model's method `show_first_layer_outputs` and a test image of your choice. **(1 point)**

- c) Use the trained U-Net classifiers and predict bone contours for the image test set `I_test` using `.predict_segmentation()` and store the predictions in `y_pred_UNet1` and `y_pred_UNet2`. Plot your predictions, the input and ground truth using `plot_prediction_triplets`. **(1 point)**

3.4 Shape Particles Filters (6 points)

Shape particle filters are sequential Monte Carlo Methods used for solving a segmentation task. Therefore a shape model is required, which subsequently is fitted to the image to be segmented. For this task we will be using the PCA shape model of the first 30 images (created in Task 1.4 of this assignment) and create a fitting routine including the definition of a cost function used in the optimization process of the fitting. The shape model will be fitted to the output of the Random Forest segmentations.

- (a) As the first step we formulate a function that models costs of fitting a shape to a target image. We are looking for a point in a parameter space (described by shape parameters, rotation, scaling, and translation) that describes an optimally fitted shape that segments the contours of a target object.

Implement the function `fit_shape_model` which should take a target image segmentation (in our case a predicted segmentation by the RF classifier in `y_pred_RF`) and the parameters `p` of the shape model as input. The implemented function should fit the shape model to the target segmentation.

- For running the optimization, use the helper function `optimize`, which takes a cost function (`cost_function`) and the upper and lower boundaries for the parameters to optimize.
- Implement the cost function which returns a scalar value that describes how well the (from `p`) generated shapes fits the classification result. (The better the shape fits the classification result, the lower the returned value). The cost function will be used by `fit_shape_model` in order to run the optimization for fitting the shape model to the segmentation.

Both `fit_shape_model` and `cost_function` will use `generate_shape` to generate shapes based on the shape parameters of the shape model and the transformation parameters used for scaling, rotation and translation. **(5 points)**

- (b) Optimize this function for all segmentations in `y_pred_RF` (20). We are using a stochastic optimization approach called **Differential Evolution**⁵ for this purpose. This method is very simple, robust and converges fast. We provide an implementation of this approach in `optimize`. An example to create and use a cost function for an optimization process can be found in `optimize_demo.py`. You can simply run it to get an example demonstration of the optimization.

⁵ http://en.wikipedia.org/wiki/Differential_evolution [last accessed 2026-05-27]

Visualize your results using the helper function `plot_fitted_shapes`. You may also plot the ground truth landmarks using this function. **(1 point)**

3.5 Interpretation Questions Classification and Feature Selection (9.5 points):

1. Qualitatively compare the images to the RF bone contour predictions and the ground truth from task 3.2 d). **(1 point)**
2. Discuss what the metrics Dice, Precision, Recall and their corresponding scores mean and what they are indicating about the model's performance from task 3.2 d). **(1 point)**
3. Evaluate and interpret the importance of different features using the helper function `show_feature_importance` in task 3.2 d). Where is the model struggling, where is it performing well? Alternate the input features and compare the results - leave features in the training out and discuss the feature importance of the random forest classifier features. **(1 point)**
4. Compare the visualized loss-graphs of your two U-Net training runs in task 3.3 a) with each other and discuss these. What is affecting them? Why are they different? **(0.5 points)**
5. Interpret the first layer output in task 3.3 b). What image features are the learned filters extracting (e.g. is the focus more on detecting edges or shapes or something else?) **(1 point)**
6. Compare the bone contour predictions `y_pred_UNet[1,2]` to the target labels `y_test` from task 3.3 c) the same way you did for the Random Forest. Evaluate and interpret the impact of (not) using data augmentation in the training process qualitatively (visualizations – you may use `plot_prediction_triplets`) and quantitatively (similarity metrics) for the test set. **(1 point)**
7. Compare the performance of the U-Net (see task 3.3) to the Random Forest (see task 3.2) using the same training and testing data split for each method. Which method performs better? Are there advantages or disadvantages? Discuss it qualitatively and quantitatively and accurately describe the experimental setup, in such a way so that it is reproducible. **(2 points)**
Hint: you can use the helper functions `plot_prediction_triplets` and `evaluate_binary_segmentation`.
8. Describe the implemented cost function for the shape particle filters from task 3.4 a). **(1point)**
9. Describe the results of shape particle filter from task 3.4 b). Where is the optimization struggling? What could be the underlying issue? **(1 point)**